

Team 1 ROB 550 BotLab Report

Pranav Jain, Austen Goddu, Jessica Huang

Abstract—This project implements and integrates the core subsystems required for autonomous navigation on the MBot platform, including motion control, simultaneous localization and mapping (SLAM), path planning, exploration, and a custom-built forklift attachment. A feedforward + PID motor controller, supplemented with gyrodometry, enables stable and accurate motion execution; mapping is achieved through a log-odds-based occupancy grid updated with LiDAR data, while Monte Carlo Localization provides robust pose estimation using an action model and a likelihood-field sensor model. These systems are combined into a full SLAM pipeline. A modified A* planner with obstacle-distance penalties generates efficient trajectories, and frontier-based exploration enables autonomous navigation of unknown spaces. Additionally, a compact rack-and-pinion forklift mechanism was designed, fabricated, and tested to support the pallet manipulation task.

I. INTRODUCTION

Autonomous mobile robots rely on a combination of motion control, perception, localization, and planning to operate effectively in real-world environments. In this project, we developed and integrated the fundamental components required for autonomous navigation on the MBot platform, including a motion controller, a SLAM pipeline, and planning and exploration modules, following techniques from Lynch and Park [1], Spong et al. [2], and Thrun et al. [3]. In addition, we also developed a forklift that would enable the robot to move and place small pallets. We implemented reliable motion control using odometry, designed a mapping and localization framework using LiDAR and particle filtering, and applied planning algorithms to support goal-driven navigation as well as autonomous exploration. This report details the methodology for each subsystem, presents experimental results, and discusses the performance and challenges of the fully integrated system.

II. METHODOLOGY

This section describes the implementation of the MBot’s motion controller, SLAM system, and path planner.

A. Motion Control

Before the SLAM or planning system can be implemented, the MBot needs a motion controller to move from pose to pose. Developing this controller consisted

of calibrating the motors, tuning a feedback/feedforward control loop for the wheel speeds, implementing odometry, and then developing and tuning a rotation, translation, rotation (RTR) motion controller.

1) *Wheel Calibration & Closed-Loop Control*: To achieve good performance with a motion control, the motors must be able to be run accurately at a commanded speed. The MBot utilizes a feedforward + PID control (FF+PID) scheme to drive the motors at the desired speed. For the feedforward controller, motor calibration was performed in order to determine the motor polarities and how various pulse-width modulation (PWM) frequencies map to actual motor speeds, both in forward and reverse. This calibration spun the robot clockwise and counter-clockwise at increasing speeds while using onboard encoders to measure the actual wheel speed. This calibration is then saved and used by the feedforward controller to determine an initial PWM frequency to match a given setpoint before refining with PID control.

2) *Motion Model & Odometry*: In order to provide an estimate of the robot’s pose without localization, wheel odometry is used following differential drive kinematics. The wheel speed is estimated using each motor’s quadrature encoder, the wheel’s radius, and the motor’s gear ratio. The estimated wheel speeds are then used to calculate the robot body velocities, $(\dot{x}, \dot{y}, \omega_{z_o})$ following equations 1 and 2, where B is the distance between the wheels, and $\dot{y} = 0$ since the robot is non-holonomic and cannot move laterally.

$$\dot{x} = \frac{\omega_{wheel_R} + \omega_{wheel_L}}{2} \quad (1)$$

$$\omega_{z_o} = \frac{\omega_{wheel_R} - \omega_{wheel_L}}{2B} \quad (2)$$

While no modifications were made to the firmware, some were made to the odometry ROS2 node on the MBot. Specifically, the odometry implementation was modified to use readings from the onboard gyroscope, using a method called gyrodometry developed by Borenstein and Feng [4]. This implementation finds the difference between the rotational velocity estimated by wheel odometry, ω_{z_o} , with the rotational velocity estimated by the gyroscope, ω_{z_g} . Based on the size of this discrepancy, the system determines whether a non-systematic error such as hitting a bump or a wheel slip has occurred, and calculates an estimated rotational velocity using more of

the gyroscope or wheel odometry's estimate accordingly. This process follows algorithm 1, where ω_{est} is the estimated angular rate of the robot's yaw. Once this rate is obtained, the position of the robot is calculated using the robot's last pose, (x, y, α_z) , the estimated linear and angular velocities, $(\dot{x}, \dot{y}, \omega_{est})$

First, if the estimated angular rate is sufficiently small, the new position of the robot (x', y', α') can be calculated according to equation 3.

$$\begin{bmatrix} x' \\ y' \\ \alpha' \end{bmatrix} = \begin{bmatrix} \cos(\alpha)\Delta t & -\sin(\alpha)\Delta t & 0 \\ \sin(\alpha)\Delta t & \cos(\alpha)\Delta t & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \alpha \end{bmatrix} \quad (3)$$

Otherwise, the instantaneous center of rotation (ICR) is used. the left and right wheel velocities are used to calculate the distance to the ICR following equation 4. The ICR location is then calculated using equation 5 and the final odometry shown in equation 6.

$$R_c = \frac{B}{2} \frac{\omega_{wheel_R} + \omega_{wheel_L}}{\omega_{wheel_R} - \omega_{wheel_L}} \quad (4)$$

$$C_r = \begin{bmatrix} x - R_c \sin(\alpha) & y + R_c \cos(\alpha) \end{bmatrix} \quad (5)$$

$$\begin{bmatrix} x' \\ y' \\ \alpha' \end{bmatrix} = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x - C_{r_x} \\ y - C_{r_y} \\ \alpha \end{bmatrix} + \begin{bmatrix} C_{r_x} \\ C_{r_y} \\ \omega_{est}\Delta t \end{bmatrix} \quad (6)$$

Algorithm 1 gyroodometry_estimate($\omega_{z_o}, \omega_{z_g}, \omega_{est}$)

```

 $\alpha \leftarrow 0.95$ 
 $\omega_{diff} \leftarrow |\omega_{z_o} - \omega_{z_g}|$ 
if  $\omega_{diff} < 0.1$  then
   $\alpha \leftarrow 0.5$ 
else if  $\omega_{diff} < 0.5$  then
   $\alpha \leftarrow 0.75$ 
 $\omega_{est} \leftarrow \alpha\omega_{z_g} + (1 - \alpha)\omega_{z_o}$ 

```

3) *RTR Controller*: To follow waypoints, the robot uses a rotation-translation-rotation controller to perform feedback control to a pose. This controller operates by first rotating to the heading of the goal point, maintaining heading and driving until the distance from the goal is zero, and then rotating to the desired orientation. Feedback control is used for this process, using a PD controller at each step. First, angular velocity ω_z is controlled according to equation 7 to rotate the robot to the heading orientation α_h . Next the robot controls both the linear and angular velocities, according to equations 7 and 8 drive to the goal position. Finally the robot controls the angular velocity according to equation 9 to rotate to the goal orientation α_g .

$$\omega_z = K_{p_1}(\alpha_h - \alpha) + K_{d_1} \frac{\alpha_h - \alpha}{\Delta t} \quad (7)$$

$$\dot{x} = K_{p_2}(dist) + K_{d_2} \frac{dist}{\Delta t} \quad (8)$$

$$\omega_z = K_{p_1}(\alpha_g - \alpha) + K_{d_1} \frac{\alpha_g - \alpha}{\Delta t} \quad (9)$$

To maintain stable driving and reduce odometry errors, two approaches were taken. First, the linear and angular velocities were limited to magnitudes of 0.2 m/s and $\frac{\pi}{4}$ rad/s, respectively. Second, low-pass filters were used to smooth the motor accelerations to prevent unwanted moments on the robot that would cause errors.

B. Simultaneous Localization and Mapping (SLAM)

1) *Mapping*: In our mapping module, we build an occupancy grid map incrementally as the robot moves through the environment. Each LiDAR scan is converted into the world frame using the robot's current pose estimate and linear interpolation, which lets us combine measurements over time in a consistent way. We use Bresenham's algorithm to find points along each LiDAR beam and mark free spaces and obstacles by decreasing and increasing the occupancy value respectively. To keep the map stable and avoid numerical issues, we use a log-odds update method, which helps the map gradually improve as more data comes in. This allows the robot to form a reliable understanding of open areas, obstacles, and unexplored areas that is later required for localization and planning.

2) *Monte Carlo Localization*: Monte Carlo localization is the method we used to localize the robot as it travels around a space. It uses a measurement model in order to simulate the actions that the robot has taken using information from the sensor model, which adds Gaussian noise to the data taken in order to model the tolerance of the sensors. By propagating particles that represent a range of possible poses from the robot through these models, we get a cloud of particles that show where the robot might currently be based on the last pose. This is then sent through the particle filter which decides from the cloud what the most likely current pose is, and the cycle repeats.

a) *Action Model*: The action model propagates the particles through the action that the robot takes. For example, if your robot drives forward 10cm and turns 90 degrees, it might have actually driven 11cm and turned 89 degrees due to imperfect actuators. This change in error is modeled in as an normal distribution that is added to our RTR model as rotation and translation error, and scaled by factors k_1 and k_2 , which we can tune. Thus, our current pose is a normal distribution of where

we're most likely to be, and the measurement model propagates the distribution through to its new shape.

To start, from odometry we have the previous and current pose:

- $(x_{t-1}, y_{t-1}, \theta_{t-1})$
- (x_t, y_t, θ_t)
- $(\Delta x, \Delta y, \Delta \theta)$

Then using geometry, we define:

$$\Delta s^2 = (\Delta x)^2 + (\Delta y)^2$$

$$\alpha = \text{atan2}(\Delta y, \Delta x) - \theta_{t-1}$$

These become rotation, translation, and rotation, which and with errors added, we have the action as:

$$\begin{aligned} \text{R: } &(\alpha + \varepsilon_1) \\ \text{T: } &(\Delta s + \varepsilon_2) \\ \text{R: } &(\Delta \theta - \alpha + \varepsilon_3) \end{aligned}$$

With Gaussian errors:

$$\begin{aligned} \varepsilon_1 &\sim \mathcal{N}(0, k_1|\alpha|), \\ \varepsilon_2 &\sim \mathcal{N}(0, k_2|\Delta s|), \\ \varepsilon_3 &\sim \mathcal{N}(0, k_1|\Delta \theta - \alpha|). \end{aligned}$$

These k_1 and k_2 values are what must be tuned in order to achieve good localization – they function as a combined system tolerance for the robot's ability to go forward or turn, but is found through guessing and checking rather than calculations. The better the odometry, the smaller the k values and the smaller the particle cloud is. Through tuning, we set our $k_1 = 0.0005$ and $k_2 = 0.01$.

Then, to apply the propagation, we finally have

$$\begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{bmatrix} + \begin{bmatrix} (\Delta s + \varepsilon_2) \cos(\theta_{t-1} + \alpha + \varepsilon_1) \\ (\Delta s + \varepsilon_2) \sin(\theta_{t-1} + \alpha + \varepsilon_1) \\ \Delta \theta + \varepsilon_1 + \varepsilon_3 \end{bmatrix}$$

to get the new pose of each particle given its old pose.

b) Sensor Model: Once the action model propagates the probability distribution to the current pose, the sensor model compares current sensor data (the map) to see where the sensors claim the robot is, and computes the probability that the robot is at each particle in the particle cloud. This becomes a fitness score that is used to weight the quality of each particle for the particle filter.

There are a couple of ways to assess the quality of a particle; we chose to implement the likelihood field model. Rather than performing computationally expensive ray tracing for each beam, the model relies on a precomputed likelihood field, which stores at every map cell the Euclidean distance to the nearest obstacle.

This enables rapid lookup of how well a beam endpoint aligns with the map.

For each beam in the scan, the measured range is first transformed from the sensor coordinate frame into the map frame through `moving_laser_scan`. This transformation accounts for the robot's estimated pose as well as any translational or rotational offset of the range sensor relative to the robot, and gives us a predicted physical location at which the beam ends. We then perform bilinear interpolation of the beam ending location using the occupancy grid cells to get an exact occupancy probability of the endpoint.

Then, for each endpoint that lies within the occupancy grid, we find the Gaussian hit probability of that beam:

$$p_{\text{hit}}(d) = \exp\left(-\frac{d^2}{2\sigma_{\text{hit}}^2}\right),$$

Because real range measurements often contain noise from reflections, outliers, or unexpected objects, we also include a uniform random component. This term assigns a small, constant probability to any measurement, regardless of its location, which acts as the unmodeled noise.

$$p_{\text{uniform}} = \frac{1}{z_{\text{max}} - z_{\text{min}}}.$$

where we set

$$p(d) = Z_{\text{hit}} p_{\text{hit}}(d) + Z_{\text{rand}} p_{\text{uniform}}.$$

where $Z_{\text{hit}} = 0.9$ and $Z_{\text{rand}} = 0.1$, which are values chosen after tuning.

Then, the likelihood of an individual beam is computed as a weighted sum of the hit and random components. Assuming conditional independence of the beams given the robot pose, the overall likelihood of the full scan is the product of the probabilities of the individual beams.

c) Particle Filter: While the whole algorithm is called the particle filter, the actual filtering is the step where given the fitness scores of all the particles in the particle cloud, an estimated robot pose is computed. There are many ways to make this estimation; however, first we resample the particles in order to prevent cloud degeneracies. From the sensor model, we have a set of particles $X_t = \{x_t^{[i]}\}_{i=1}^M$ with corresponding normalized weights $W_t = \{w_t^{[i]}\}_{i=1}^M$. The objective is to replicate particles in proportion to their weights while reducing the randomness inherent in standard resampling.

First we draw a single random offset $r \sim \mathcal{U}(0, M^{-1})$, which determines the starting point of M equally spaced sampling positions along the cumulative weight distribution. For each of the M output samples, a target location is computed as

$$U_m = r + (m - 1)M^{-1}, \quad m = 1, \dots, M.$$

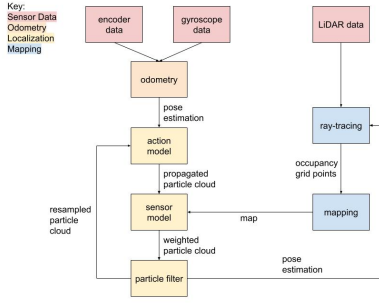


Fig. 1: SLAM block diagram

The algorithm then steps through the weight array while maintaining a cumulative weight counter c . For each sampling location U_m , particles are advanced until the cumulative weight first exceeds the threshold U_m . The corresponding particle $x_t^{[i]}$ is then selected as the m -th element of the resampled particle set.

Because particles with larger weights occupy larger segments of the cumulative distribution, they are selected more frequently, while particles with very small weights may not be selected at all. Since the sampling points are deterministically spaced rather than independently drawn, this reduces the variance of the number of times each particle is selected and thus prevents particle degeneracy, improving the stability of the particle filter.

Once this is done, we can finally take our posterior pose estimate based on this resampled cloud. We initially simply tried taking the weighted average of all of the particles, using their fitness scores as weights. When our localization struggled, we then switched to using the weighted average of all the particles that had at least 90% of the weight of the top weighted particle. However, this did not seem to make a large impact on the quality of our localization.

3) *Combined Implementation (SLAM)*: Once our mapping and localization worked independently, we combined them for SLAM. This was a simple task — we just wired the map generated from our mapping to our localization as the known map to use for the sensor model. Then our estimated pose would be wired back so that the map could use it to update the map, as shown in figure 1.

4) *Kidnapped Robot Problem*: Due to time constraints, a solution to the kidnapped robot problem was planned but not implemented. The planned solution is described here. First, since the robot could be anywhere in a known map, the initial particles in the particle filter would be initially sampled from a uniform distribution across the map’s known free areas. Next, the robot would execute a small spin, while letting the particle filter run and letting the particle distributions travel. Finally, it

would attempt to navigate from its estimated pose to the home position to clear any remaining uncertainty.

C. Planning and Exploration

Once the SLAM is complete, the next steps are to to path plan and explore so that the robot can autonomously map its surroundings and make its way to set goal points.

1) *Path Planning*: After obtaining the occupancy map, the A* search algorithm is applied for path planning. A* combines Dijkstra’s algorithm with the Greedy Best-First Strategy by minimizing a cost function composed of the actual path cost and a heuristic term. The Manhattan-based heuristic is used, while 8-neighbors allow both grid and diagonal motion. The cost of moving horizontally or vertically is set to 1, and diagonal motion is assigned a cost of $\sqrt{2}$, enabling more direct paths. To discourage trajectories from passing close to obstacles, an additional penalty term is incorporated:

$$\text{penalty} = 100 \cdot \frac{1}{\text{dist}_{\text{obs}} + 10^{-6}},$$

where dist_{obs} denotes the distance to the nearest obstacle. This encourages the search to remain in regions that maintain a safe clearance. The addition of 10^{-6} in the denominator prevents numerical issues arising from division by zero. After a path is produced by A*, it contains many short, unnecessary intermediate points. To address this, a line-of-sight-based pruning method is employed. Starting from the first waypoint, the algorithm checks whether a straight-line connection to a later waypoint remains collision-free according to the distance grid. Any intermediate points along such segments are removed, yielding a smoother trajectory suitable for execution.

2) *Exploration*: Frontier-based exploration is used to guide the robot through unknown areas of the environment. Frontiers are defined as free cells that border at least one unknown cell in the occupancy grid. To identify these regions, the algorithm iterates through the entire grid and applies `isFrontierCell`, which checks that whether a cell is free and that one of its four neighboring cells remains unobserved. Each frontier cell is then converted into world coordinates through `gridToWorld` and stored as a potential exploration point.

Because raw frontier points are often noisy and spatially fragmented, the system groups them into clusters before computing exploration goals. To do this, each frontier point is mapped back into grid coordinates and a breadth-first search (bfs) is performed using an 8-connected neighborhood. The `findCluster` routine expands each frontier region by iteratively adding neighboring frontier cells that have not yet been visited. Only

clusters above a minimum size threshold are retained to eliminate noisy or isolated cells.

After clustering, the centroid of each frontier region is computed by averaging the world-frame positions of all frontier cells within that cluster. These centroids serve as candidate exploration targets. Before selecting a goal, the candidates are filtered based on clearance provided by the obstacle distance grid: any centroid whose corresponding grid cell has insufficient obstacle distance is discarded. The remaining candidates are then ranked by their Euclidean distance from the robot's current pose, with the closest valid centroid selected as the next exploration goal. A minimum distance threshold is also applied to avoid selecting frontiers that lie too close to the robot and are, therefore, uninformative.

D. Forklift

The forklift was designed to pick up pallets with 4 inch cubes and stack them on top of each other. In addition, it needed to not obstruct the front camera so that the robot could see AprilTags. The result was a simple rack-and-pinion design that uses the XL-320 servos to drive the forklift up and down, as shown in figure 3. The forklift features four prongs and bumps on the bottom of the prongs to avoid getting caught on bumps on the floor while driving, which are featured in figure 2. The racks are held in place by the servo mounts, which feature a protruding fixture that slots into the rack, holding it in place. This innately also provides some hard stops so that the rack cannot drive too far up or down (though in the down case, the floor also works to hard stop). The Bill of Materials can be found in the Appendix.

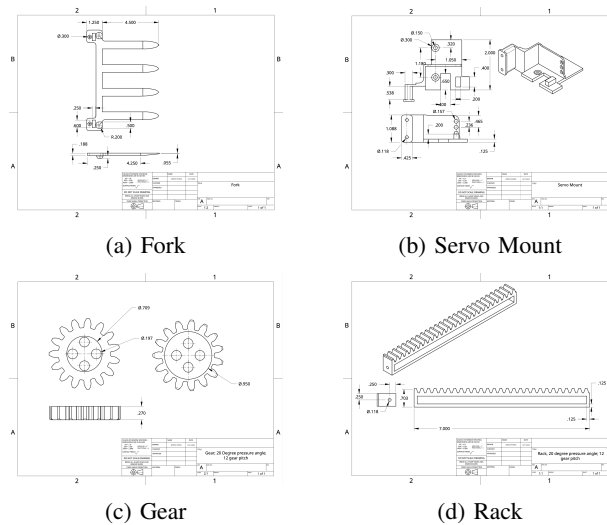


Fig. 2: Part drawings for each main part of the forklift.

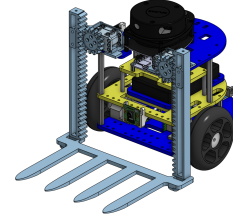


Fig. 3: CAD of the forklift design.

To assemble the forklift, first mount the servo mounts to the top layer of the robot using bolts. Then mount the servos as shown using the plastic rivets. Then again mount the gears to the servos using plastic rivets. At this point, you can mount the racks to the forklift using bolts. Finally, you can mount the rack-forklift assembly by carefully pulling open the rack slot to fit around the lip of the servo mount fixture and aligning the gear and rack teeth. This part is tricky so it is important to be careful – it helps to install the racks halfway up because that is where the slot can flex open the widest. Once the rack-forklift assembly is installed, you can drive the forklift up or down.

Once the system is installed, you can wire the servos as instructed in the XL-320 setup instructions. Be sure to not daisy chain them and instead wire them independently. This will ensure that there can be no significant voltage drop between the servos that might impact servo behavior.

We did not have time to implement servo code, but here is how we would have done it:

The servos would have their own address and can be driven independently, but in order to drive the forklift up or down, they need to drive at the same RPM in opposite directions. Luckily, this can be done easily by simply setting the RPM of each servo independently. This works when the servos are not daisy chained because they both get the power they need to maintain the same RPM, but when they are daisy chained, one servo could eat up the power and the other ends up turning significantly slower, causing the forklift to bind.

III. RESULTS

A. Motion Control

1) *Motor Calibration:* The slope, y-intercept, and variance for the motor calibration, run three separate times can be seen in table I. Overall, we see consistent means and low variance for both motors, with lower variances on the right motor. This variance can come from a number of systematic and non-systematic sources, such as backlash in the motor gearboxes, or dirt on the wheels and floor causing variable friction.

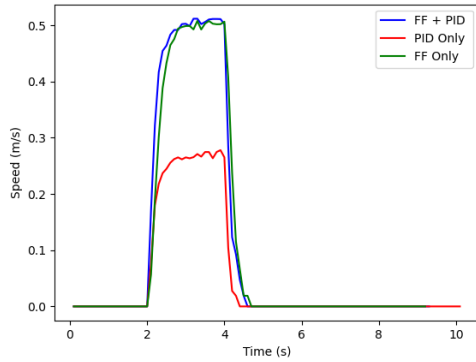


Fig. 4: PID, Feed-Forward, and Combined control methods for a 0.5 m/s step command plotted together

2) *PID Tuning*: Figure 4 compare three wheel-control strategies responding to a 0.5 m/s step input: feedforward (FF) only, PID only, and FF+PID. The FF+PID controller exhibits the best overall behavior: it reaches the setpoint rapidly (short rise time), exhibits minimal overshoot, and leaves near-zero steady-state error, combining the responsiveness of feedforward with the steady-state correction of feedback. The FF-only controller exhibits the worst overall behavior: it reaches the setpoint quickly but shows measurable steady-state error due to friction. The PID-only controller removes steady-state error but is slower and can show greater overshoot and settling time when gains are increased. Some improvements may include implementing anti-windup for the integral term, filtering the derivative term, refining the feedforward model to account for static friction, and running the controller at a consistent high update rate.

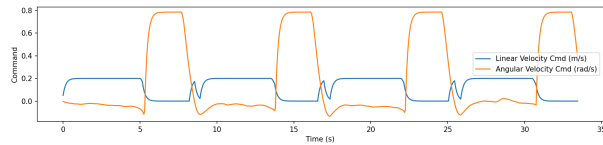


Fig. 5: Linear and angular velocities of the robot while completing one loop around the 1 m square.

3) *Motion Controller*: Figure 5 illustrates the robot's linear and angular velocities during a single 1 m square

	Left Motor (μ, σ^2)	Right Motor (μ, σ^2)
Positive Slope	(5.96E-02, 4.30E-07)	(5.39E-02, 1.98E-08)
Positive Intercept	(4.98E-02, 8.01E-05)	(5.94E-02, 6.45E-08)
Negative Slope	(5.66E-02, 1.05E-07)	(6.32E-02, 1.24E-09)
Negative Intercept	(-5.56E-02, 0.08E-06)	(-4.31E-02, 8.94E-07)

TABLE I: Motor Calibration - Mean and Variance of Positive and Negative Slope and Intercept values across three calibration runs

loop. The linear velocity remains nearly constant along the straight segments, while the angular velocity peaks sharply at the corners where the robot needs to turn. This pattern demonstrates that the motion controller effectively accelerates and decelerates the robot to follow the square trajectory, ensuring smooth transitions between straight and rotational movements. Minor oscillations in the linear velocity at the corners indicate the controller's adjustment to maintain accurate heading while turning.

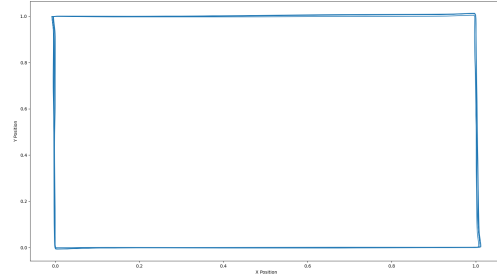


Fig. 6: Estimated robot trajectory over four consecutive 1 m square loops.

Figure 6 shows the robot's estimated pose over four consecutive laps. The robot maintains a close approximation of the intended square path, demonstrating accurate gyrodometry and effective motion control. Minor deviations accumulate over repeated loops, highlighting the expected drift when relying solely on odometry without SLAM corrections.

B. Simultaneous Localization and Mapping (SLAM)

1) *Mapping*: Initially, the mapping node relied solely on gyrodometry, which caused the map quality to deteriorate over time as the robot's estimated pose drifted from its true position. As a result, later portions of the map became increasingly misaligned with earlier iterations.

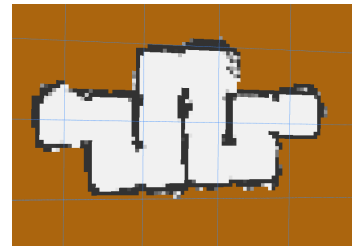


Fig. 7: Map generated using only gyrodometry.

Figure 7 illustrates this cumulative drift, with the map visibly degrading toward the end of the trajectory.

2) *Monte Carlo Localization*: We were able to run our localization with a range of 100-1500 particles, and

for our SLAM we opted for 1500 particles for best results. As shown in Table II, the frequency decreases with particle number; we were asked to estimate 10 Hz but decided to manually test it instead and found that 1200 particles runs at about 10Hz

Particle #	Time (s)	Hz
100	0.008	125
500	0.043	23
1000	0.081	12
1200	0.094	10
1500	0.125	8

TABLE II: Time to update the particle filter for each number of particles.



Fig. 8: Localization with 1500 particles, comparing the estimated pose path (green) with the reference pose path (blue).

The comparison in Fig. 8 shows that the Monte Carlo Localization system with 1500 particles is able to track the robot’s ground-truth trajectory throughout the environment with reasonable accuracy. Although the estimated pose path (green) follows the overall shape and structure of the reference trajectory (blue), a noticeable constant offset is present along most of the trajectory. This offset arises from small errors in the initial pose estimate, which the particle filter maintains consistently rather than eliminating entirely. Despite this translational shift, the relative motion is captured accurately. Minor deviations also occur in narrow corridors and around tight turns, where limited LiDAR visibility introduces additional uncertainty, but the filter stabilizes as measurements accumulate. Overall, the localization system demonstrates stable and drift-free relative tracking, and the remaining offset reflects an initialization bias rather than runtime divergence—making the results suitable for mapping and planning tasks where relative consistency is more critical than exact global alignment.

3) *Combined Implementation (SLAM)*: When the mapping was performed using only gyrodometry, the resulting map degraded over time, as seen previously in Figure 7.

After integrating localization with mapping, the SLAM implementation produced a map that remained

stable and consistent throughout the entire run. Because the robot’s pose was regularly corrected using LiDAR observations, drift no longer accumulated, and the map retained its global alignment even after long trajectories.

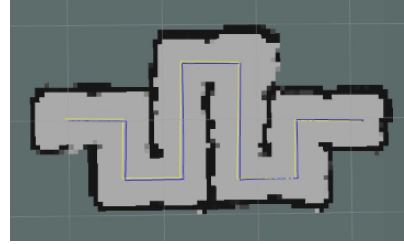


Fig. 9: Map generated using SLAM. The yellow path is the ground-truth, and the blue path is path taken due to gyrodometry.

Figure 9 shows the resulting SLAM-based map. In contrast to the gyrodometry-only map, the SLAM map preserves straight walls, maintains correct alignment when revisiting earlier areas, and avoids the large distortions seen previously. Overall, the SLAM approach results in a significantly clearer and more accurate representation of the environment.

C. Path Planning

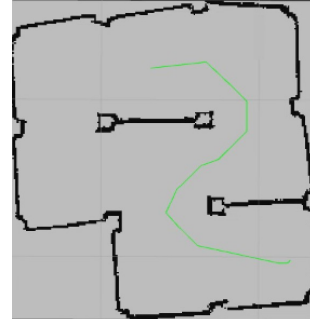


Fig. 10: Path planning on a pre-generated map. The bottom-right corner is starting point and top-center is the goal.

Figure 10 shows the resulting path generated by the A* planner on a pre-generated occupancy grid map. The robot starts in the bottom-right corner and successfully computes a collision-free route to the goal located near the top-center of the map. The planner selects diagonal motions when beneficial and keeps a safe distance from nearby obstacles due to the applied distance-based penalty. The final path is relatively smooth, demonstrating that the planner can produce efficient and reliable trajectories with an accurate map.

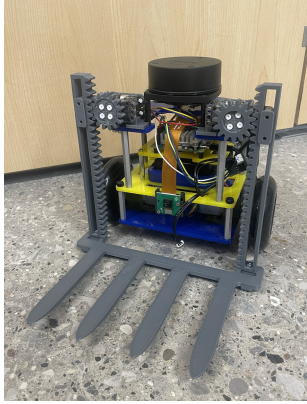


Fig. 11: A photo of the forklift, assembled.

D. Forklift

The forklift performed overall relatively well, being able to drive smoothly up and down manually. There was mild cantilevering due to the lack of support behind the forklift, but not enough to cause problems, just some chafing on the wheels.

The assembly was moderately difficult, with the majority of it being the slightest bit inconvenient, like trying to attach the rack to the fork when the space in the slot to reach the nut or bolt was rather small, or the need to simultaneously open the slot and mesh the rack and gear at the same time. Technically, this could be avoided by installing the rack before the gear, but we found that it was difficult to remove the gear once it was installed, which meant that if we wanted to remove the forklift and then reinstall it, this would be the best method. In figure 11, one of the gears is installed backwards by mistake but not correct due to the difficulty of removing the plastic rivets. However, this had no impact on the performance of the forklift.

E. Competition

There are three events for this competition: Accuracy and Speed Runs, Factory Patrol Challenge, and Warehouse Challenge. Due to time limitations, we were unable to participate in the second and third events.

In the accuracy run of Event 1, the robot relied exclusively on gyrodometry to reach each of the commanded poses. Despite the inherent drift associated with wheel encoders and gyroscopes, the robot maintained strong short-term pose consistency throughout the run, allowing it to follow the instructed trajectory with minimal cumulative error. This is reflected in the final pose accuracy, where the robot stopped within 3cm in x, 3cm in y, and 1° in heading of the ground-truth target. These results highlight that, although gyrodometry alone is not sufficient for long-duration navigation or exploration

tasks, it provides stable and reliable local odometry over the timescale of the Event 1 trial. The robot was also able to generate a good map. It was able to complete the same course in 82 seconds for the Speed Run, earning 200/300 points for Event 1.

IV. DISCUSSION

The MBot's motion controller maintained stable linear velocities and precise turns, while gyrodometry reduced rotational errors. SLAM integration eliminated drift, enabling accurate global mapping and efficient A* path planning. Monte Carlo localization provided consistent pose estimates, with minor offsets from initialization bias. The forklift functioned reliably, though assembly complexity and slight cantilevering were observed. Overall, the system demonstrates effective coordination of perception, control, planning, and manipulation. Future work could address the kidnapped robot problem, optimize particle filter performance, and improve forklift stability and assembly.

V. CONCLUSION

Through the development of a complete navigation and perception stack, we equipped the MBot platform with the ability to autonomously map, localize, plan, and manipulate objects within its environment. The motion controller, combining feedforward and PID control with gyrodometry enabled the robot to accurately execute planned trajectories. By integrating Monte Carlo Localization with occupancy-grid mapping, we produced a SLAM system that eliminated long-term drift and maintained globally consistent maps, enabling reliable downstream planning. The A* planner, enhanced with obstacle-distance penalties and waypoint pruning, generated smooth, safe paths, while frontier-based exploration expanded the robot's ability to autonomously navigate unfamiliar spaces. Finally, the forklift mechanism enabled manual pallet lifting and stacking, despite minor structural flex and assembly challenges.

REFERENCES

- [1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*, 1st ed. Cambridge University Press, May 2017. [Online]. Available: <https://www.cambridge.org/core/product/identifier/9781316661239/type/book>
- [2] M. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Wiley, 2005. [Online]. Available: <https://books.google.com/books?id=wGapQAAACAAJ>
- [3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. The MIT Press, 2006. [Online]. Available: <http://www.probabilistic-robotics.org/>
- [4] J. Borenstein and L. Feng, "Gyrodometry: a new method for combining data from gyros and odometry in mobile robots," in *Proceedings of IEEE International Conference on Robotics and Automation*, vol. 1. Minneapolis, MN, USA: IEEE, 1996, pp. 423–428. [Online]. Available: <http://ieeexplore.ieee.org/document/503813/>

APPENDIX

Bill of Materials:

Item	Quantity	Description
PET-G Plastic	As needed	Filament
4-40 Nuts	6	Standard steel
1/4" 4-40 Bolts	6	Machine screws
M3 Plastic Rivets	14	Snap rivets
Dynamixel XL-320 Servo	2	Servo

TABLE III: Bill of Materials